IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 27 February 2026, accepted 11 March 2026, date of publication 23 March 2026, date of current version 6 April 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3676577

RESEARCH ARTICLE

Evaluating Retrieval-Augmented Generation for LLM-Based Vulnerability Detection: An Empirical Study on Real-World Java Vulnerabilities

GÁBOR ANTAL[1,2], LÓRÁNT ÉRTEKES[1], NORBERT SZOLNOKI[1], AND PÉTER HEGEDŰS[1]
[1]Department of Software Engineering, University of Szeged, 6720 Szeged, Hungary
[2]FrontEndART Software Ltd., 6721 Szeged, Hungary

Corresponding author: Gábor Antal (antal@inf.u-szeged.hu)

This work was supported in part by the Ministry of Culture and Innovation of Hungary from the National Research, Development and Innovation Fund, through the K_23, the Hungarian Scientific Research Fund/Országos Tudományos Kutatási Alapprogramok (OTKA) Funding Scheme, under Project K 147225; in part by European Union Horizon Program under Grant 101120393 (Sec4AI4Sec); and in part by the University of Szeged Open Access Fund under Grant 8297.

ABSTRACT Software vulnerabilities are growing as fast as the digital platforms and applications that contain them. Thus, the timely and effective detection of software vulnerabilities is becoming increasingly important. Another emerging trend is the widespread use of large language models (LLMs) for software engineering tasks such as source and test code generation, refactoring, and debugging. Finding security-related issues is crucial, but it is a very resource-intensive task. This work offers an empirical benchmark of the impact of retrieval-augmented prompts on LLM-based vulnerability detection, rather than proposing a new detection method. In this work, we explore the source code vulnerability detection capabilities of large language models (LLMs) in a realistic, language-specific setting. We investigate to what extent retrieval-augmented generation (RAG) improves their performance when applied to real-world Java vulnerabilities. We evaluated seven widely used LLMs (GPT-5, GPT-4o, Claude Sonnet 4, Gemini 2.5, LLaMa 4, DeepSeek 3.1, and Grok Code Fast) on Java source code. We used the Vul4J dataset, a manually curated benchmark of real-world vulnerabilities, comparing a basic zero-shot approach against a RAG-enhanced approach that provides up to three vulnerable and three secure code examples based on their semantic similarity to the code under analysis. To mitigate the non-deterministic nature of LLMs, each experiment was repeated three times and the results were averaged. We empirically found that RAG helps LLMs reduce false positives without significantly affecting recall, leading to improved MCC scores under specific conditions. However, retrieval was often sparse, and false positives remained common, limiting practical impact. Our study serves as an empirical evaluation rather than a deployment-ready methodology.

INDEX TERMS Vulnerability detection, large language models, retrieval-augmented generation, LLM comparison.

I. INTRODUCTION

Detecting and repairing software vulnerabilities as early as possible in the development cycle is an important goal in software engineering. Uncaught weaknesses can lead to serious losses if exploited by malicious actors. No wonder that automatic vulnerability detection (AVD) is an area that has been extensively researched. There are multiple different approaches to AVD, such as traditional static code analysis tools [15], [27] and deep learning based methods [26], [42], [54], which are emerging directions in this field. Large language models (LLMs) have advanced to a point where they became interesting for the vulnerability detection task as well. They show strong code comprehension abilities, which encouraged researchers to study their performance in vulnerable code detection and repair [2], [17], [22], [31], [48],

The associate editor coordinating the review of this manuscript and approving it for publication was Tyson Brooks.

[52], [53]. Due to promising results, the body of research on the use of LLMs for AVD is continuously growing. Many of these experiments apply various techniques in an attempt to further enhance the vulnerability detection abilities of LLMs, such as prompt engineering [6], [53], fine-tuning [17], [31], [48], or retrieval-augmented generation (RAG) [12], [19], [49]. Some studies integrate LLMs with static code analysis tools to examine whether LLMs can complement them to catch bugs missed by them or to reduce false alarms made by them [25], [45].

This study intentionally focuses on Java, which remains one of the most widely used languages in enterprise and security-critical systems (According to surveys such as the TIOBE index,[1] or the StackOverflow Survey[2]). Several well-curated benchmarks, such as Defects4J or Vul4J, are available exclusively for Java. Our aim is not to study cross-language generalization, but to evaluate whether retrieval-augmented prompting can meaningfully improve LLM-based vulnerability detection in a realistic, language-specific setting, where subtle semantic differences between vulnerable and fixed code are common.

Our study contributes to this line of research by empirically evaluating the vulnerability detection capability of seven state-of-the-art LLMs under comparable conditions. We also carefully constructed a retrieval-augmented database to assess the extent to which providing additional contextual examples improves model performance. Rather than proposing a new detection method, our work provides a systematic analysis of how recent LLMs behave when distinguishing between vulnerable and fixed real-world Java code. We evaluated more recent LLMs than most related studies [2], [53]. We also thoroughly checked our RAG database for overlaps with the test data and described the applied mitigation methods in detail. Since our test data consisted of vulnerable–fixed code pairs, we additionally evaluated pairwise detection metrics, which remain underexplored in prior work. Finally, we applied ensemble techniques to estimate the upper bound of performance that can be achieved by combining multiple models.

We emphasize that this work does not propose a new detection methodology, but rather offers an empirical evaluation of retrieval-augmented prompting applied to recent LLMs in a practical code vulnerability detection setting. While our experiments show that RAG can improve performance metrics in some cases, key limitations such as false positives and sparse retrieval coverage remain unresolved. These practical constraints are discussed in detail in Section V.

The research aims to answer the following questions:

- **RQ**$_1$: How do state-of-the-art LLMs compare in their effectiveness for vulnerability detection tasks?
- **RQ**$_2$: To what extent does retrieval-augmented generation (RAG) enhance the vulnerability detection capabilities of LLMs?
- **RQ**$_3$: To what extent do ensemble approaches outperform individual models?

The paper is structured as follows. In Section II, we overview the related literature. We present our methodology in Section III. The main findings of our study can be found in Section IV. The possible threats to our study's validity are discussed in Section V. We conclude the paper and outline future research directions in SectionVI.

II. RELATED WORK

Vulnerability detection has an extensive literature. Starting from static [4], [29], [47] and dynamic analysis [3], [50] (and of course their combination [1], [34]), this field has evolved a lot in recent years. Early methods treated for vulnerability detection treated it as a supervised classification problem, training models on labeled vulnerable and secure code examples. Traditional deep learning approaches [8] ranged from code similarity analysis and clone detection (e.g., VulDeePecker [26]) to graph neural networks that learn program semantics (e.g., Devign [54]).

The use of large language models for vulnerability detection has gained a lot of attention in recent years. VulDeePecker [26] is considered one of the first studies on this topic that includes deep learning to detect vulnerabilities. The transformer revolution fundamentally changed the field, starting with the introduction of CodeBERT [13], and its variants DistilBERT [37] and RoBERTa [28]. Since their introduction, several papers have used CodeBERT for vulnerability detection with various success [23], [24], [30], [46], [51]. Le et al. [24] used CodeBERT and GPT fine-tuning, with the base CodeBERT models achieving F1 scores between 0.25-0.43, while the various GPT models (base and fine-tuned) achieved F1 scores between 0.29-0.44. Risse and Böhme [35] demonstrate that state-of-the-art ML4VD techniques severely overfit to unrelated features when evaluated using semantic-preserving code transformations. They tested six leading models (UniXcoder, CoTexT, GraphCodeBERT, CodeBERT, VulBERTa, PLBart) on CodeXGLUE/Devign, they show that while training data augmentation restores 69.0% of accuracy lost to testing transformations, augmenting training data with different transformations than testing data causes additional performance drops of 30.2% (CodeXGLUE) and 77.5% (VulDeePecker). The authors introduce VulnPatchPairs, a dataset containing 26,200 vulnerable C functions paired with their patches, revealing that all six models achieve only 43.4% average accuracy when distinguishing vulnerabilities from patches, despite baseline accuracies of 63-69% on standard data.

The emerge of CodeT5 [43], [44] introduced identifier-aware pre-training with unified encoder-decoder architecture. Chen et al. [9] presented a study along with their dataset called DiverseVul. Their study evaluated 11 deep learning models from 4 families (GNN, RoBERTa, GPT-2, and T5) for vulnerability detection in C/C++ code. In some cases, large language models significantly outperformed the state-of-the-art GNN-based ReVeal model, with the best performing

[1] https://www.tiobe.com/tiobe-index/
[2] https://survey.stackoverflow.co/2025/technology/

models being NatGen (F1 score: 0.4715), CodeT5 Base (F1 score: 0.4569), and CodeT5 Small (F1 score: 0.4510). However, a critical finding was the poor generalization to unseen projects, where F1 scores dropped dramatically to only 0.094 on unseen projects.

Numerous studies exist on fine-tuning LLMs as well; however, in this paper we only consider studies that are similar to ours; other studies can be found in relevant surveys and literature review [14], [20], [52].

Retrieval-augmented generation is an emerging paradigm shift in vulnerability detection, which addresses serious limitations of the use of LLMs by integrating external knowledge bases, vulnerability data and historical patterns to improve detection accuracy and reasoning capabilities. Du et al. [12] presented Vul-Rag, a knowledge-level RAG framework for vulnerability detection. This paper addresses a critical limitation of LLMs in vulnerability detection: their inability to distinguish between vulnerable code and similar but secure code, given the often subtle differences between them. Originally, they achieved only 6-14% pair accuracy (meaning they correctly identify both vulnerable and fixed code in the same pair). They built a multi-dimensional vulnerability knowledge representation using e.g., historical Common Vulnerabilities and Exposure (CVE) patterns and extracted data from 1,154 CVEs (2,317 vulnerable-patched pairs). The authors built a three-stage workflow and used their own built dataset, however, exhaustive overlapping detection is not performed. In their paper, they included four well-known state-of-the-art LLMs, including GPT-4o, Claude Sonnet 3.5, Qwen2.5-Coder-32B-Instruct, and DeepSeek-V2-Instruct. In our paper, we use almost all of these models, however, we use the more recent versions of them. In this study, we did not include any smaller models. Their results showed that they were able to improved pair accuracy by +16% to +24% (our results can be found in Section IV).

Safdar et al. [36] proposed Real-VulLLM, a comprehensive evaluation framework to assess LLMs capabilities in detecting and reasoning about software vulnerabilities in real-world scenarios. In their study, the authors tests both zero-shot and few-shot (with context) scenarios. They evaluated 5 state-of-the-art LLMs: GPT-4, Gemini-1.5-Flash, Qwen2.5-Coder, LLaMA-3, and Phi-4. They reported a gain of more than 49% in correct predictions and a 35% decrease in incorrect predictions metrics, in the best case.

SecLLMHolmes [40] is an automated framework by Ullah et al. This study evaluates whether LLMs can detect security vulnerabilities, testing 8 models across 228 code scenarios, covering 8 critical vulnerability types. The best-performing model, GPT-4, achieved 89.5% maximum accuracy on hand-crafted examples but performed significantly worse on real-world CVEs, with all models showing high false positive rates by incorrectly flagging patched code as vulnerable. Even advanced models like GPT-4 and PaLM2 proved non-robust in this task, with simple code changes (renaming variables/functions, adding whitespace) causing incorrect answers in 17-26% of cases. The authors conclude that LLMs are not yet reliable for real-life vulnerability detection.

Unlike the above listed prior works that primarily focuses on prompt engineering strategies, fine-tuning, or synthetic vulnerability benchmarks, our study aims to empirically characterize the limits of retrieval-augmented prompting when applied to real-world vulnerable-fixed code pairs. In particular, we focus on false positive behavior and pairwise discrimination between vulnerable and patched code.

III. METHODOLOGY

Our goal was to compare multiple state-of-the-art LLMs in terms of their ability to classify vulnerable and secure Java source codes via binary classification. We were also curious whether and to what extent retrieval-augmented generation (RAG) improves their performance.

RAG is a technique that helps LLMs by providing relevant context data in the prompt, in our case, examples of similar vulnerable and secure source code snippets. Thus, we first built a database of such labeled codes. We chose 4 publicly available datasets on the internet, processed and standardized them, and removed entries that overlapped with our test data. Then, we created semantic embeddings for all the entries using OpenAI's *text-embedding-3-small* embedder model and stored them in ChromaDB,[3] a vector database. The semantic embedding of a text is a multi-dimensional vector representing its meaning. Ideally, if we take two pieces of text, the more similar their meaning is, the closer their embedding vectors should be, measured using e.g., euclidean distance. These embeddings are important because the retriever component of the RAG system can find more similar, thus more relevant pieces of code to provide as context to the code in question.

A. TEST DATA SELECTION

We used a subset of the Vul4J [7] dataset as our test data, a total of 72 pairs of vulnerable-fixed codes. Vul4J is a dataset that contains 79 real-world Java vulnerabilities from 51 open-source projects. Each vulnerability includes the vulnerable code, the human-written fix that mitigates it, and one or more *Proof of Vulnerability* test cases that demonstrates the security flaw in action. The dataset covers 25 different types of security weaknesses. It comes with a framework that lets users checkout, compile, and test these vulnerabilities with ease. The dataset is designed to help researchers develop and evaluate automated tools that can detect and fix security bugs in Java code.

The primary reason for our choice was that Vul4J is a reliable, manually curated, cleaned dataset. We would like to note that we did not use the fix or the PoV unit test in the present study, but we intend to utilize them in future research.

[3]More information can be found at https://docs.trychroma.com/.

B. RAG DATA SOURCE SELECTION

Our RAG database contains data from 4 datasets available online: CVEFixes dataset [5], CyberNative Vulnerability dataset [11], Shestov et al. LLM Fine-tune dataset [38] and MegaVul [31]. We chose these datasets, because the source codes were readily available for download in CSV format, without having to run any source code mining script to get them (thus preventing a possible threat to our dataset's construct validity).

The CVEfixes dataset [5] contains Common Vulnerabilities and Exposures (CVE) records from the public U.S. National Vulnerability Database (NVD). It has more than 31 thousand entries in many different programming languages and more than 1,000 of them are written in Java. The Java subset of the dataset has a vulnerable-to-secure code snippets ratio of roughly one-to-one.

The Cybernative.ai Code Vulnerability and Security Dataset [11] is available on Hugging Face[4] and is used by many researchers to train their models on. This is an artificial dataset, generated using a *Deepseek Coder* [16] based LLM. It is made up of more than 4,600 pairs of vulnerable-fixed code pairs in various languages, more than 400 of which are written in Java.

The LLM Fine-tune dataset created by Shestov et al. [38] contains code snippets written in Java, partly derived from the CVEfixes dataset and partly from mining vulnerability fixing commits from real-world projects. The authors of the dataset state that their dataset has 2 versions; they refer to them as "X1 without P3" and "X1 with P3". The former has 1,334 samples and has a ratio of vulnerable to secure codes of approximately 1:1. The latter has 22,945 samples with a ratio of about 1:34. Although the latter dataset with far more instances belonging to the secure class would be closer to the real-world distribution of vulnerabilities, training an LLM on such an imbalanced dataset leads to far worse model performance than training it on the balanced former one, the authors concluded. Based on this, to optimize performance, we decided to keep the positive to negative example balance close to 1:1 in our RAG dataset, and so we used the "X1 without P3" version of this dataset.

The MegaVul dataset [31] was constructed by mining vulnerability fixing commits from Git repositories. Functions that have been changed during such commits were extracted, their pre-commit versions marked as vulnerable, and their post-commit versions marked as fixed. Functions not changed during these commits were labeled non-vulnerable. In terms of language, the first version of the dataset contained C/C++ codes, but later they also made a Java version. Our study focuses on Java, so we took the latter version. This contains 2,433 pairs of vulnerable-fixed codes and 39,516 non-vulnerable functions. Similarly to how we treated the Shestov et al. dataset, we only put the vulnerable-fixed pairs into our RAG database and ignored the additional non-vulnerable functions, again to keep the balance of positive and negative classes. Table 1 shows the count of vulnerable and secure records for all the different datasets we used for our RAG database.

[4]https://huggingface.co/, A platform providing open-source machine learning models and datasets.

TABLE 1. RAG data sources.

Dataset name	Vulnerable Records	Secure Records	Total
CVEFixes[5,6]	581	581	1162
CyberNative[7]	424	424	848
Shestov et al.[8]	665	669	1334
MegaVul[9]	2433	2433	4866
Total	**4103**	**4107**	**8210**

C. RAG DATA PROCESSING

Our research focused on Java, thus we filtered out non-Java records from the mixed language datasets CVEfixes and CyberNative. The datasets by Shestov et al. and MegaVul could be split into a balanced part and a secure records only part. The balanced part of the datasets contains about the same number of vulnerable and secure records. In our study, we only used the balanced parts for the RAG database. This left us with a total of 8,210 records. We manually read a random sample of codes from all four datasets and observed multiple problems. First of all, we found syntactically incorrect codes in all four datasets used. We also noticed that in some cases, code comments contained hints about the vulnerability status of the code. This would be especially problematic, if these codes would be used as test data, but even for use in the RAG database, we decided to remove comments to make sure that the LLM relies purely on the code itself and will not be influenced by non-functional parts of it. Lastly, indentation lengths, the order of modifiers, annotations and whether the code is wrapped into a class or interface also varied across datasets, which could potentially affect the LLM.

Based on these findings, our next goal was to rigorously check, process, reformat and (where necessary and possible) repair syntactically erroneous records, to ensure that the data we will use for RAG is of high quality and as standard as possible. We used JavaParser,[10] a publicly available Java library that can parse, transform, and generate Java code. It also helped us in detecting syntactically incorrect codes, remove all comments and reformat the codes to achieve standard indentation lengths and a fixed order of modifiers and annotations. We tried to repair syntactically wrong records detected by JavaParser. In the CVEfixes dataset,

[5]https://zenodo.org/records/7029359

[6]https://www.kaggle.com/datasets/girish17019/cvefixes-vulnerable-and-fixed-code

[7]https://huggingface.co/datasets/CyberNative/Code_Vulnerability_Security_DPO

[8]https://github.com/rmusab/vul-llm-finetune

[9]https://github.com/Icyrockton/MegaVul

[10]https://javaparser.org

we had a total of 84 issues, but none of them could be repaired, so we had to exclude them. All codes in the CyberNative dataset were wrapped with three backticks (```). This resulted in a syntax error while trying to parse them, thus we unwrapped the code inside. After this, 270 records still failed the syntax check. The majority of these syntax errors were caused by comment-like texts, which were present in the code without being commented out. Removing these texts solved the issue. There were 7 records however, which could not be repaired this easily, because they contained unfinished methods, thus we had to omit them. The codes in the Shestov et al. and MegaVul datasets, unlike those in CVEfixes and CyberNative, consisted of bare methods, they were not wrapped inside a class or interface. This caused them to fail the syntax checks of JavaParser. To solve this, we wrapped them with a class definition, or if they contained default methods, then with an interface definition (using "SomeClass" and "SomeInterface" as names). After adding the class or interface definition, all codes passed the syntax check. After solving all syntax errors, JavaParser removed all comments and successfully reformatted the codes.

D. OVERLAP CHECK

At first, we started experimenting using the current state of our RAG database. As we manually evaluated the results of these preliminary runs using the prompt with RAG, we encountered multiple cases where the LLM answered that the vulnerability status of the code in question is obvious, as it was already provided in the context part of the prompt. We checked the prompt and the code in question was indeed part of the example codes whose vulnerability status is given. The observed problem was due to overlaps between the RAG database and the test dataset (despite the fact that the test data came from a totally different source than our RAG data). This issue might also affect other studies that use RAG, especially if they did not conduct an extensive overlap check.

To filter out overlapping records between the RAG database and the test dataset, we employed a longest common subsequence (LCS) [32] based method. The *length of the LCS* (LLCS) of two strings is a simple but effective measure of their similarity. The LLCS value of two strings can range from zero to the length of the shorter string. This is an absolute measure, depending on the length of the compared strings. To make it a relative measure, with a value ranging from zero to one, we need to divide the LLCS by the maximum value it can reach, which is the length of the shorter string. The problem with this metric is that it could reach values close to one even when the compared strings are not particularly similar, e.g., when one string is contained within the other. In our case, the length of codes varies heavily, therefore, we decided to counteract such potential false positives by dividing the LLCS by both the length of the shorter string and the length of the longer string, then taking the arithmetic average of the two quotients. To calculate the LCS based syntactic similarity score between two source codes, we used Formula 1.

$$\mathrm{sim}(c_1, c_2) = \frac{1}{2}\left(\frac{|\mathrm{LCS}(c_1, c_2)|}{|c_1|} + \frac{|\mathrm{LCS}(c_1, c_2)|}{|c_2|}\right) \quad (1)$$

The result of this formula gives us a real number between 0 and 1, i.e. the similarity ($\mathrm{sim}(c_1, c_2)$ between code snippet 1 (c_1) and code snippet 2 (c_2). Zero indicates that there is no single shared character between c_1 and c_2, 1 indicating that the two codes are identical.

We calculated this LCS based syntactic similarity score for all possible RAG code-test code pairs. We then manually evaluated the results, starting from code pairs showing the highest similarity. We saw many examples where a test method (only the method itself, without class definition) was found in a RAG database code. Based on this pattern, we decided to check for overlaps using another method, too. This method involved removing the class definition from the test code, eliminating all whitespace characters from both the RAG code and the test method, and then using string matching to determine whether the whitespace-free and class-definition-free test method could be found within the whitespace-free RAG code. Using this method, we discovered that 70 of our 144 test records appeared identically in some of the datasets used for RAG. Based on these results, we removed 7 records from the CVEfixes dataset, 58 records from the Shestov et al. dataset, and 23 records from the MegaVul dataset, resulting in a total of 88 records removed from our RAG database. Furthermore, we manually looked at all the RAG code-test code pairs that have survived the string-matching-based overlap check and that have achieved at least 0.65 LCS based syntactic similarity score on the LCS based overlap check. We decided to remove 11 records from the Shestov et al. dataset and 9 records from the MegaVul dataset. These records were not exactly just almost identical to certain test records, that is why they were not caught by the string matching based overlap check. To sum up the results of the two different overlap check methods, we had to remove a total of $88 + 20 = 108$ records from our RAG database, which made it shrink to a size of 8,011 records.

E. PROMPT CREATION

To create our prompts, we needed a RAG database to store and retrieve data. We chose to use ChromaDB, a widely known open-source vector database. First, we had to choose an embedding model for our case. We used an OpenAI embedder model called *text-embedder-3-small* which is used by many researchers and practitioners [18], [21].

Embeddings are multidimensional vectors that contain real numbers that represent the semantic meaning of a text. A measure of such vectors distance widely used for embeddings is the calculation of the cosine of the angle between two vectors, which is often referred to as *cosine similarity score*, which was also used in our study.

Our prompt incorporated code snippets retrieved from the RAG database that are semantically similar to the code under classification, providing the LLM with relevant examples for

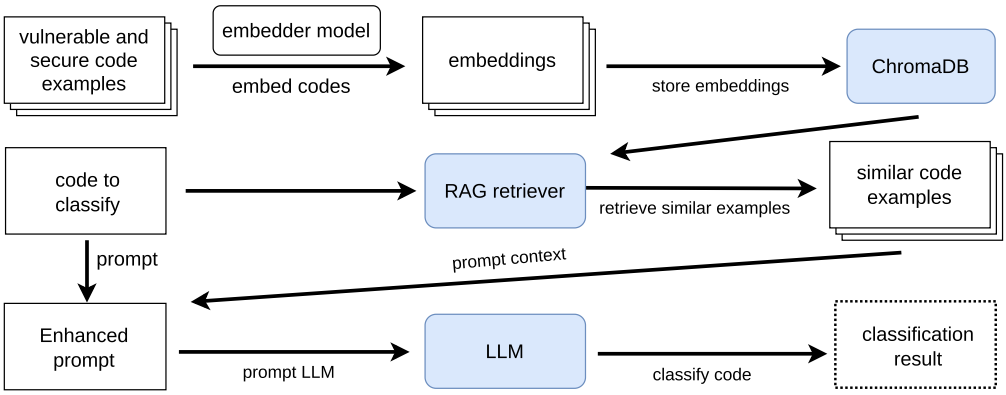


FIGURE 1. A flow chart of our RAG enhanced prompt creation process.

classifying the code as vulnerable or secure. As a preliminary research, we experimented with 3 different prompts on 15 vulnerable-secure code pairs, and then we chose the best performing one for our research. The vulnerability status of the retrieved code snippets is also indicated in the prompt, so basically we show the LLM a few examples of vulnerable code snippets and a few examples of secure code snippets semantically similar to the code under classification.

We then constructed two variants of our prompt: a zero-shot version without examples and a few-shot version augmented with retrieval-augmented generation (RAG). The latter contains at most three vulnerable and at most three secure source code examples retrieved based on semantic similarity. The retrieved vulnerable and secure code examples are independent. They are not the vulnerable and fixed versions of the same code. The number of retrieved vulnerable and secure code examples are also independent, thus it is possible to retrieve, for example, two vulnerable and no secure code examples for a given test code. An example prompt is shown in Figure 2, where only one similar vulnerable snippet and two similar secure snippets were available. To better signal the beginning and the end of code blocks, we framed every code snippet in the prompt with three backticks (```). After the opening backticks we also wrote "java" to clarify the programming language of the code shown. We indicated the vulnerability status of each code example by adding the line "this code is vulnerable:" or "this code is secure:" before the code block. Zero-shot prompts naturally omit the "Context" section but are otherwise identical to the few-shot variant in all other aspects. Code snippets were retrieved using cosine similarity, with a minimum similarity threshold of 0.6 to the code under classification. Studies that conducted empirical experiments testing multiple minimum similarity thresholds have concluded that values around [33] or exactly [10] 0.6 led to the best results. Importantly, we do not require the model to name or classify a specific vulnerability type; a prediction is considered correct if the model correctly identifies the presence or absence of a vulnerability in the code, independent of whether it articulates the exact underlying weakness. To provide a better overview of our RAG enhanced prompt creation process, a flow chart is shown in Figure 1.

Please answer the question below based on the following context.
Context:
Similar Vulnerable Code | Similar Secure Code | Similar Secure Code | Similar Secure Code
Question: You are a source code security expert that is good at static program analysis.
Please analyze the following code:
Code in Question
Please indicate your analysis result with one of the options:
(1) A vulnerability detected.
(2) The code is secure.
Only reply with one of the options above.

FIGURE 2. A schematized example prompt what we used.

F. STUDY EXECUTION

We evaluated 7 different LLMs in this study: *GPT-5, GPT-4o, Claude Sonnet 4, Grok Code Fast 1, Gemini 2.5 Flash, Llama 4 Maverick,* and *DeepSeek 3.1*. Most of these models were ranked in the Top 10 in the OpenRouter programming category as of September 22, 2025. Llama 4 Maverick and GPT-4o were the only exceptions (both can be found in the Top 25 models), which we wanted to include because these models have been extremely widely used in the field up to the time of writing this study.

Once the preparations were completed, we could begin running the experiments. We ran all seven models with both the RAG-enhanced prompts containing examples and the zero-shot prompts without any examples. Every single example (e.g., *VUL4J-3, vulnerable, RAG prompt*) was executed independently from others. To reduce the non-determinism

nature of the models, we performed the entire run three times. We conducted 3 runs with all seven models on the 72 vulnerable-secure pairs, yielding a total of 3,024 results.

Despite the prompt, the models did not always respond with a single word. These cases were then processed separately using a very simple rule-based evaluator that flagged problematic cases even when the slightest ambiguity occurred. These were reviewed by two authors who independently evaluated them. Then they were verified with the participation of a third participant, resulting in the final values derived from the model responses.

IV. RESULTS

A. OVERVIEW

We ran our experiments with and without retrieval assisted generation, for all seven chosen LLMs. This is what we considered to be one batch, and we ran a total of three batches. We stored the predicted labels extracted from the responses of the LLMs separately for the zero-shot and the RAG enhanced prompt and for each of the 3 runs. Thus, we created a total of $2 \times 3 = 6$ result tables. Each of these tables contained 72 rows, since there are 72 pairs of test data, and 2 columns for each of the 7 models. The first column of each model contained its predicted labels for the vulnerable codes, while the second column contained those for the secure codes.

We counted true positive, true negative, false positive and false negative results within each table, separately for each model. In our case, the positive class means vulnerable, while the negative class means secure source codes. Based on these counts, we then assessed the false positive rate (or type 1 error), false negative rate (or type 2 error), accuracy, and Matthews Correlation Coefficient (MCC) within each table, separately for each model. The false positive rate measures the ratio of false positive predictions to the total count of secure records. Similarly, the false negative rate measures the ratio of false negative predictions to the total count of vulnerable records. The accuracy is the ratio of successful to all predictions. The value of the Matthews Correlation Coefficient ranges from -1 to 1, 1 meaning a perfect classification, -1 meaning a perfect inverse classification, and 0 indicating no better than random performance.

We also calculated precision, recall, and F1-scores within each table, separately for each model and vulnerability class. The recall for the positive class is also called the true positive rate, while the recall for the negative class is also called the true negative rate. The true positive rate measures the ratio of vulnerable test data that the LLM predicted as positive. The true negative rate reveals the ratio of secure test data that was predicted as negative. The precision for the positive class tells us what ratio of the LLMs positive predictions was right, while the precision for the negative class shows the ratio of negative predictions being right. The F1 score aggregates the recall and precision in a balanced manner, calculating the harmonic mean of them separately for the positive and negative classes.

Finally, we calculated the arithmetic mean for each of the previously calculated metrics across the 3 runs, separately for the zero-shot and the RAG prompt, thus creating two tables that contain aggregated results per prompt type.

B. METRICS EVALUATION

Table 2 shows the aggregated results for the aforementioned metrics for the prompt without RAG, while Table 3 shows this for the prompt with RAG. False positive rates declined on average by 12.54% (e.g. 7.80 percentage points) due to RAG, GPT-4o, Gemini-2.5-Flash, DeepSeek-3.1 and Llama-4 Maverick dropping it by more than 16%, GPT-5 and ClaudeSonnet-4 by more than 9%. Looking at precision and F1 scores, RAG improved them on average in both the vulnerable and secure classes, models GPT-4o and Gemini-2.5-Flash gaining the most. Comparing accuracies, RAG helped the models Gemini-2.5-Flash, GPT-4o and Llama-4 Maverick the most, increasing their accuracy scores by 14.55%, 9.09% and 5.43% (e.g. 7.41, 5.09 and 3.24 percentage points), respectively. Except for the Grok-Code-Fast-1 model, all other models benefitted from RAG in terms of accuracy, on average by 4.95% (e.g. 2.68 percentage points). Similar observations can be made about MCC scores, all but the Grok-Code-Fast-1 model benefited from RAG. The average MCC increased by 40.7% (e.g. 4.99 percentage points) due to RAG, Gemini-2.5-Flash, GPT-4o and Llama-4 Maverick leading the list again with increases of 14.82, 10.06 and 5.89 percentage points, respectively.

We aggregated the count of correct classifications across all 3 runs and all 7 models for each of the 144 test records and separately for each of the 2 prompts. This way, we were able to analyze the effect of the RAG enhanced prompt at the test record level. Unfortunately, in 89 cases, the RAG enhanced prompt did not include any vulnerable or secure examples, because there were no codes in the RAG database that had at least 0.6 similarity scores with these test records. In these cases, the RAG enhanced prompt was the same as the zero-shot prompt. In the other 55 cases, the RAG prompt contained at least one vulnerable or at least one secure example. Of these 55 cases, there were 36 cases where the count of correct classifications did not change, 10 cases where it decreased, and 9 cases where it increased. Of the 10 cases where it decreased, there were 4 cases where it decreased by more than 2, and of the 9 cases where it increased, there were 5 cases where it increased by more than 2. These results show that the effect of the RAG enhancement varies much depending on the test record.

To further investigate the reason why the RAG enhanced prompt improves detection performance for some test records in comparison to the zero-shot prompt but does not help or even worsen performance for other records, we also looked at distributions of seven pernicious kingdoms (7PK) categories [39]. The 7PK category represents a broader vulnerability type. The only significant pattern we observed

TABLE 2. Arithmetic average of metrics from 3 runs, using the prompt without RAG.

Metric	GPT-5		GPT-4o		ClaudeSonnet-4		GrokCodeFast-1		Gemini-2.5-Flash		DeepSeek-3.1		Llama-4 Maverick	
	Vulnerable	Secure	Vulnerable	Secure	Vulnerable	Secure	Vulnerable	Secure	Vulnerable	Secure	Vulnerable	Secure	Vulnerable	Secure
FPR	65.74%		62.04%		81.02%		53.24%		50.93%		68.98%		55.09%	
FNR	22.69%		25.93%		8.33%		30.56%		47.22%		25.93%		25.46%	
Recall	77.31%	34.26%	74.07%	37.96%	91.67%	18.98%	69.44%	46.76%	52.78%	49.07%	74.07%	31.02%	74.54%	44.91%
Precision	54.06%	60.13%	54.43%	59.41%	53.09%	69.39%	56.65%	60.41%	50.96%	50.89%	51.75%	54.76%	57.47%	63.99%
F1-score	63.62%	43.60%	62.74%	46.32%	67.23%	29.81%	62.38%	52.68%	51.83%	49.93%	60.91%	39.54%	64.88%	52.73%
Accuracy	55.79%		56.02%		55.32%		58.10%		50.93%		52.55%		59.72%	
MCC	12.81%		12.91%		15.47%		16.63%		1.85%		5.75%		20.43%	

TABLE 3. Arithmetic average of metrics from 3 runs, using the prompt with RAG.

Metric	GPT-5		GPT-4o		ClaudeSonnet-4		GrokCodeFast-1		Gemini-2.5-Flash		DeepSeek-3.1		Llama-4 Maverick	
	Vulnerable	Secure	Vulnerable	Secure	Vulnerable	Secure	Vulnerable	Secure	Vulnerable	Secure	Vulnerable	Secure	Vulnerable	Secure
FPR	59.26%		51.39%		73.61%		54.17%		41.20%		56.94%		45.83%	
FNR	24.54%		26.39%		12.50%		32.41%		42.13%		37.04%		28.24%	
Recall	75.46%	40.74%	73.61%	48.61%	87.50%	26.39%	67.59%	45.83%	57.87%	58.80%	62.96%	43.06%	71.76%	54.17%
Precision	55.96%	62.67%	58.88%	64.85%	54.32%	67.83%	55.49%	58.66%	58.40%	58.27%	52.51%	53.75%	61.16%	65.54%
F1-score	64.26%	49.35%	65.42%	55.57%	67.02%	37.97%	60.93%	51.43%	58.13%	58.53%	57.26%	47.80%	66.01%	59.25%
Accuracy	58.10%		61.11%		56.94%		56.71%		58.33%		53.01%		62.96%	
MCC	17.38%		22.96%		17.53%		13.79%		16.67%		6.14%		26.31%	

regarding this metric was that the ratio of test data belonging to the input validation and representation category was less than 50% among those where the RAG enabled prompt helped, about 50% where the RAG prompt performed just as well as the zero-shot and about 80% where the RAG prompt underperformed the zero-shot. Based on these observed patterns, RAG enhancement may be less useful in improving vulnerability detection performance for vulnerabilities related to input validation and representation.

We also searched for patterns related to patch length. We measured the length of patches by the number of lines changed between the vulnerable and secure versions of the same test data. To calculate this metric, we used Myers' algorithm, which returns the minimum number of additions and deletions required to transform the vulnerable code into the secure one. After calculating the patch length for each test record, we discretized these results into three categories. We considered patches of length less than or equal to 5 lines to be short patches, those above 5 but less than or equal to 10 lines to be medium long patches, and those above 10 lines to be long patches. The ratio of short patches was between 75 and 80% in all three groups, but the ratio of long patches shows visible differences between the three groups, it was greater than 10% in the groups where RAG enhancement improved or did not change detection performance, but there was not a single example of a long patched record in the group where the RAG enhanced prompt worsened the performance. This pattern might indicate that RAG enhancement is more likely to improve vulnerability detection performance for vulnerabilities with long patches.

RQ$_1$: How do state-of-the-art LLMs compare in their effectiveness for vulnerability detection tasks?

The seven LLMs showed varying performance levels. Without RAG, accuracies ranged from 50.93% to 59.72%, with Llama-4 Maverick having the best accuracy and a Matthews Correlation Coefficient (MCC) of 20.43%. The FPR from the vulnerability detection point of view ranged from 50.93% to 81.02%. ClaudeSonnet-4 achieved the highest recall, but it had the biggest false positive rate. All things considered, the high false positive rate makes LLMs, in their current form, unsuitable for vulnerability detection on their own.

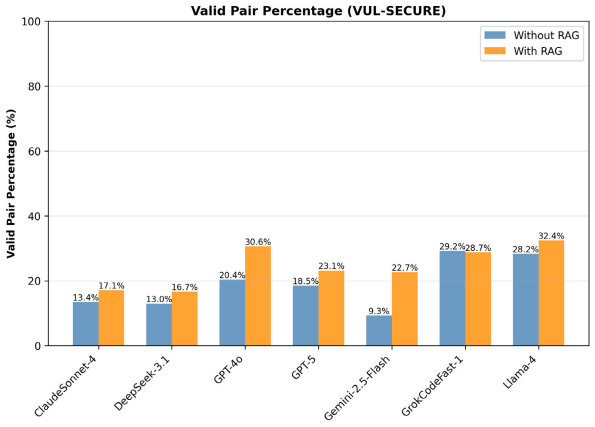


FIGURE 3. Percentage of pairs where both vulnerable and secure methods were correctly classified.

C. PAIRWISE RESULTS (PAIR ACCURACY)

Besides traditional performance measures, one major quality indicator of the vulnerability detection approaches is their ability to distinguish between vulnerable and fixed version of the same source code. Studies often analyze the vulnerability detection performance of tools on unrelated vulnerable and non-vulnerable code snippets. This can be misleading as empirical works, for example, that of Viszkok and Hegedus [41] who evaluate SAST and ML-based vulnerability detection tools using the OSSF CVE benchmark[11] and show that they are prone to detecting the same vulnerability even

[11] https://github.com/ossf-cve-benchmark/ossf-cve-benchmark

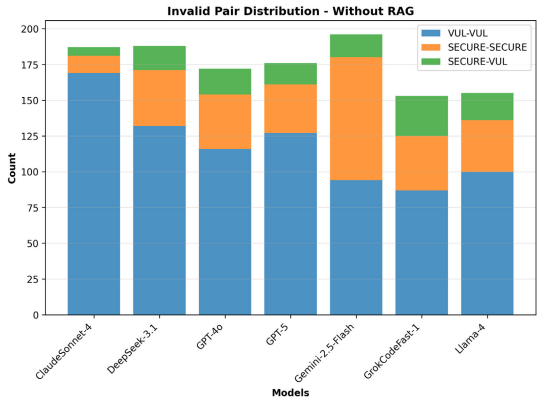


FIGURE 4. Count of pairs where at least one of the methods was incorrectly classified using the prompt without RAG.

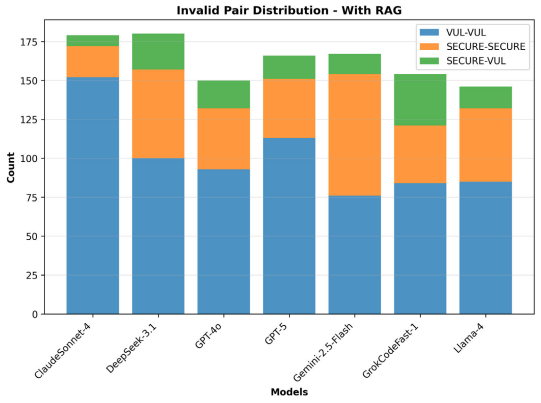


FIGURE 5. Count of pairs where at least one of the methods was incorrectly classified using the prompt with RAG.

after applying a correct fix to the source code. Such tools cannot be considered reliable since they cannot properly distinguish between the vulnerable and fixed version of the same code pattern, yielding to potential false alarms. Moreover, this behavior may indicate a deeper problem with the detection tool itself as it might have detected an issue based on spurious correlations or the wrong/inaccurate code pattern.

Therefore, we analyze the so-called ''pairwise'' detection performance of the LLMs, which refers to their capability of detecting the disappearance of a properly detected vulnerability after its fix (i.e., we expect that a vulnerable version of the code is detected as vulnerable but at the same time, its fixed version as free from vulnerabilities). The test data we used for the evaluation consisted of vulnerable-fixed source code pairs, which allowed us to measure the pairwise performance of LLMs. Note that these code pairs typically differ only slightly as many of the vulnerability fixes consist of minor code modifications. Thus, the LLMs had to grasp these tiny, but from a security point of view crucial distinctions.

We measured the count of code pairs, where both the vulnerable and the secure version of the code was correctly classified across all batches per model and per prompt. Figure 3 shows the ratio of correctly classified pairs compared to all pairs in all batches per model and per prompt. The ratio of pairs correctly classified was 5.61 percentage points higher on average with RAG. Models Gemini-2.5-Flash and GPT-4o experienced an increase of more than 10 percentage points. The Grok Code Fast 1 model was the best performing without RAG, correctly classifying 29.2% of the pairs, but with RAG, Llama 4 Maverick and GPT-4o outperformed it by correctly classifying 32.4% and 30.6% of the pairs, respectively.

This result highlights the positive impact of our RAG approach on LLM vulnerability detection performance in terms of pairwise detection. However, LLM-based vulnerability detection (even enhanced with RAG) is not free from this mislabeling effect. As can be seen in Figures 4 and 5, the number of invalid pairs is still high, among which the VUL-VUL category forms the majority. This means that models tend to produce a lot of correct predictions on vulnerable code but fail to detect those as non-vulnerable after their repair. Using RAG examples reduces this effect, as shown in Figure 5. Misclassified pairs where LLMs predict both the vulnerable and non-vulnerable versions to be secure form the second highest group, while it is very rare that models predict vulnerable code as secure and its non-vulnerable counterpart as vulnerable. This effect also manifests in the high *FPR* values, showing that LLMs are much prone to mislabeling non-vulnerable code, even if they correctly identified their vulnerable state, primarily due to subtle fix semantics, conservative model behavior, and limited retrieval coverage.

RQ_2: To what extent does retrieval-augmented generation (RAG) enhance the vulnerability detection capabilities of LLMs?

RAG significantly improved detection performance across most models. False positive rates decreased by an average of 12.54% (from 62.43% to 54.63%). Accuracy increased by approximately 4.95% on average (about 2.68 percentage points), ranging from 53.01% to 62.96%. The best individual improvement happened with the Gemini-2.5-Flash model, which improved from 50.93% to 58.33% (+7.41 percentage points, meaning a 14.55% increase). MCC also improved improved by a significant 41% on average.

Pair accuracy (correctly classifying both vulnerable and fixed versions of the same entry) improved by 5.61 percentage points on average. For example, Gemini-2.5-Flash increased from 9.3% pair accuracy to 22.7% having the relative improvement of more than 140%. Best performing model (in terms of pair accuracy) was the Llama-4 Maverick model peaking at 32.4% pair accuracy. We would like to note however, even with these improvements, the testes LLMs have quite high false positive rates, meaning that their usability in real-life is limited.

D. ENSEMBLE TECHNIQUES

We were curious about what results we could achieve if we combined multiple models instead of a single model,

and whether the combined results would help or hinder detection effectiveness. For this experiment, we used the already existing results from all seven models, both with and without RAG. We labeled each different run with numbers, but beyond identification, the numbers have no significance. For example, "GrokCodeFast-1(b1)" means that we used the *GrokCodeFast-1* model, without any additional RAG examples, from the first run, while "GPT-5(RAG b3)" means we used the *GPT-5* model with RAG from the third run.

During the experiment, we combined the results through simple majority voting. We calculated the results for combinations of 3 and 5 model results (resulting in a total of 862,148 possible ensemble models). We also did a little experiment on some ensemble models consisting of 7 model results (the total possible combinations would have been more than 25 million). However, these did not produce any outstanding results, so in the study we only used ensemble models consisting of 3 and 5 models.

Table 4 presents the top 10 ensemble models ranked by their accuracy in vulnerability detection, showing F1 scores for both vulnerability and secure code classification as well.

The Top 10 ensemble models achieved accuracies between 69.44% and 70.83%. This suggests that the top-performing combinations reached similar effectiveness levels, with the best model only marginally outperforming the others. The F1 scores show an interesting pattern, as vulnerable code classification (F1-Vulnerable is between 0.72 and 0.75) consistently outperforms secure code classification (F1-Secure is between 0.63 and 0.67). This suggests that (ensemble) models find it more challenging to correctly identify secure code, possibly due to the subtle differences in secure and vulnerable codes. Interestingly, all top 10 ensemble models use RAG-enhanced models. For example, Llama-4 (RAG) appeared in every top-performing ensemble model, suggesting that this model with RAG examples provided important contributions to the ensemble predictions. The best individual model accuracy from Table 3 was 62.96% (Llama-4 with RAG), while the best ensemble achieved 70.83%, which is an improvement of approximately 7.87 percentage points (12.50%). This shows that the ensemble methods can successfully improve the results of individual models.

Figure 6 shows the accuracy distribution across all possible ensemble models (862,148 total). The best ensemble model achieved an accuracy of 70.83%, while the worst-performing model reached only 40%. Interestingly, most ensemble models achieved worse results than the best-performing individual model (which has an average accuracy of 62.96%). According to our results, only 40,579 (4.71%) ensemble models outperformed the best individual model, while 821,569 (95.29%) ensemble models could not exceed the performance of the best individual model.

It is worth noting that even the best ensemble accuracy of 70.83% still leaves substantial room for improvement, and it is achieved by combining the results of several models, which makes this method hard to apply in real-life.

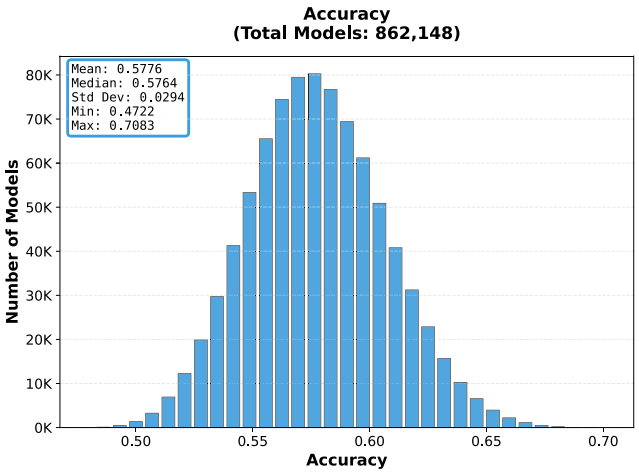


FIGURE 6. Accuracy distributions across all ensemble models.

RQ_3: To what extent does ensemble approaches outperform individual models?

Ensemble methods demonstrated improvements over individual models. The best ensemble model achieved 70.83% accuracy, which is a 12.50% improvement (7.87 percentage points) over the best individual model (Llama-4 Maverick with RAG at 62.96%). However, only 4.71% of the 862,148 tested ensemble models outperformed the best individual model. All top 10 ensemble models used RAG-enhanced versions and included Llama-4(RAG) in their combinations, suggesting careful model selection is critical for ensemble success.

V. THREATS TO VALIDITY

There might be some factors that threatens our study's validity. In this section, we list them and describe what we have done to mitigate them.

A. INTERNAL VALIDITY

We have written and used many scripts to explore, filter and process the datasets we used, to set up the RAG framework, including embedding and uploading the RAG records to ChromaDB and retrieving relevant codes to assemble the prompts, and lastly to automatically evaluate as many of the answers of the LLMs as possible. There is a risk that some of our codes were written incorrectly and thus did not function as intended. To mitigate this risk, we performed a thorough code review on all of our scripts.

To reduce the risk that automatic evaluation incorrectly classifies results, we used a strict set of rules. It only classified a result as class A if it contained a clear indication of class A, and at the same time it did not contain even a slight indication of class B. Otherwise, it just flagged the result to be manually evaluated.

Label noise is another threat to our study. Especially an incorrect labeling of the test dataset would impact the validity of our results. To keep this risk under control, we chose the Vul4J dataset for evaluation. It is a reliable manually curated

TABLE 4. Top 10 ensemble model performers.

Model	F1-Vul	F1-Sec	Accuracy
ClaudeSonnet-4(RAG b2) + GPT-5(RAG b3) + GrokCodeFast-1(RAG b3) + Llama-4(RAG b2+RAG b3)	0.75	0.65	0.7083
GPT-4o(RAG b1+b3) + GrokCodeFast-1(b1) + Llama-4(RAG b2+RAG b3)	0.73	0.66	0.7014
ClaudeSonnet-4(RAG b2) + GPT-4o(b1) + GPT-5(RAG b3) + Gemini-2.5-Flash(RAG b1) + Llama-4(RAG b3)	0.74	0.64	0.7014
GPT-4o(RAG b1) + GrokCodeFast-1(RAG b3+b1) + Llama-4(RAG b2+RAG b3)	0.73	0.67	0.7014
DeepSeek-3.1(b3) + GPT-5(RAG b3) + GrokCodeFast-1(b2) + Llama-4(RAG b2+RAG b3)	0.73	0.65	0.6944
ClaudeSonnet-4(RAG b1) + GPT-4o(b1) + GPT-5(RAG b3) + Gemini-2.5-Flash(RAG b1) + Llama-4(RAG b3)	0.74	0.63	0.6944
GPT-5(RAG b3) + Gemini-2.5-Flash(RAG b1) + GrokCodeFast-1(b1) + Llama-4(RAG b2+RAG b3)	0.72	0.66	0.6944
GPT-5(RAG b3) + GrokCodeFast-1(RAG b3+b2) + Llama-4(RAG b2+RAG b3)	0.72	0.66	0.6944
ClaudeSonnet-4(RAG b2) + DeepSeek-3.1(b3) + GPT-5(RAG b3) + Llama-4(RAG b2+RAG b3)	0.74	0.63	0.6944
GPT-5(RAG b3) + Gemini-2.5-Flash(RAG b1) + GrokCodeFast-1(b2) + Llama-4(RAG b2+RAG b3)	0.72	0.66	0.6944

dataset with human-written fixed methods and one or more proof of vulnerability tests to prove the vulnerable nature of the methods labeled vulnerable.

An overlap between the RAG database and the test dataset is also a major risk to our experiment. We responded to this kind of threat by applying an extensive overlap check. We checked every possible RAG data-test data combination using two different methods, one of them used string matching and returned a binary answer, while the other one used a longest common subsequence based similarity measure and returned a continuous value between zero and one. We removed every record from the RAG dataset where the former method found a match and then manually evaluated all pairs that were not overlapping according to the former method, but had an LCS based syntactic similarity score of at least 0.65 according to the latter method. Manual evaluation led to the removal of more RAG records, ones that were too similar to some of the test data.

The answers of LLMs can be nondeterministic, which poses another relevant threat to our findings. To counteract this issue, we increased the sample size by doing a total of 3 runs from every prompt and LLM combination.

B. EXTERNAL VALIDITY

External validity indicates the extent to which the findings of a study can be generalized to different contexts. We used seven different LLMs, and the Vul4J dataset we employed for evaluation contains 25 distinct types of security weaknesses. This level of variety provides a reasonable degree of representativeness for our target domain. However, the study is limited to a single programming language (Java), which restricts the generalizability of the findings. Vulnerability types and coding idioms differ across languages, and it remains to be seen whether similar improvements would be observed for other languages such as C/C++, Python, or JavaScript. Repeating the study in other programming languages is an important direction for future work. Java-specific factors such as small, localized fixes and API-centric vulnerability patterns may influence false positive behavior, while the main effects of retrieval-augmented prompting are expected to generalize across languages and evaluation benchmarks.

The evaluation set includes 72 vulnerable-fixed code pairs. While this sample size imposes limitations on statistical power, the dataset offers practical relevance, as the examples are drawn from real-world vulnerabilities with human-written patches. The fix patterns range from small local edits to more extensive structural changes. This makes the test set particularly suitable for assessing whether LLMs can distinguish between insecure and secure code, and for analyzing false positive tendencies.

Despite these strengths, several constraints affect the broader applicability of our results. One such constraint is the persistence of false positives. Even when retrieval-augmented prompts were used, models often failed to recognize that a vulnerability had been correctly fixed. In many cases, both the vulnerable and fixed versions of a code pair were classified as vulnerable. This limits the real-world applicability of these models, particularly in settings where high false positive rates could reduce the trust and utility of automated vulnerability detection tools.

Another limitation arises from the retrieval step of the RAG process. Due to the cosine similarity threshold of 0.6, approximately 61% of the test cases did not retrieve any examples from the RAG database. In these cases, the prompt reverted to a zero-shot format, which may have reduced performance consistency. This highlights the sensitivity of retrieval-based augmentation to the distribution and coverage of the embedding space, as well as the quality and density of the support dataset.

Privacy concerns also arise due to the use of remotely hosted LLMs, because data has to be transmitted through the internet. Thus, in case of sensitive data, this approach cannot be used. Alternatively, locally hosted open source models could be used in such cases, but they have considerable hardware requirements, and their performance might not be comparable.

We also note that the non-deterministic nature of LLMs introduces further variability into the results. Although we repeated every experiment three times and averaged the outcomes to mitigate this effect, model responses can still be unstable across runs. During manual inspection, we observed occasional hallucinated or unjustified explanations, which may affect reproducibility and reliability. These characteristics reinforce the need for caution when interpreting results

and suggest that further refinement of both prompt structure and model control mechanisms is needed.

VI. CONCLUSION

This paper investigated whether Retrieval-Augmented Generation can enhance the effectiveness of Large Language Models in detecting real-world software vulnerabilities. We evaluated seven state-of-the-art LLMs using Java source code from the Vul4J dataset, a manually curated Java vulnerability dataset. We compared zero-shot results against RAG-enhanced results that provided similar vulnerable and secure code examples.

Our findings demonstrate that RAG can improve vulnerability detection performance across multiple metrics. On average, RAG reduced false positive rates by 12.54% (from 62.43% to 54.63%), increased accuracy by 4.95% (approximately 2.68 percentage points), and improved Matthews Correlation Coefficient by 40.7% (4.99 percentage points). The pairwise detection analysis revealed that RAG improved the models' ability to distinguish between vulnerable and fixed versions of the same code by 5.61 percentage points on average. This is particularly relevant for practical applications, as the ability to recognize when a vulnerability has been properly fixed is critical for reducing false alarms in real environments.

However, these gains remain conditional and limited. In over 60% of cases, the retriever failed to return relevant examples due to the cosine similarity threshold, leading the model to revert to zero-shot prompting. Additionally, models frequently misclassified fixed code as still vulnerable, indicating persistent false positives even with RAG enhancement. These limitations constrain the practical effectiveness of the approach.

Given these limitations, we see the practical use of RAG-enhanced LLMs not as standalone vulnerability detectors, but as complementary decision-support tools within existing security workflows. In particular, they can assist in reducing false positives, providing contextual explanations, and supporting triage or prioritization of findings produced by traditional static analysis tools, for example when integrated into CI pipelines or security code review processes. Before such systems could be reliably deployed, several conditions would need to be met, including substantially lower false positive rates, improved retrieval coverage to ensure relevant examples are consistently available, and more stable model behavior across runs. Additionally, evaluation on larger and more diverse vulnerability datasets would be necessary to demonstrate robustness in real-world environments.

Our experiments on ensemble approaches showed that combining multiple models through majority voting can further improve results, with the best ensemble achieving 70.83% accuracy (a 12.50% improvement over the best individual model). Our research opens several promising directions for future investigation; one of them being the multi-language evaluation and comparison, using popular programming languages such as C/C++, Python, or JavaScript, to achieve a general overview. Future research could investigate whether using additional context in RAG prompts (e.g., tests, project-specific guidelines) improves detection accuracy and reduces false positives. Combining fine-tuning approaches with RAG techniques could potentially increase performance. Investigating the synergy between these two enhancement strategies would be a promising research direction. These avenues could help bridge the gap between current LLM capabilities and the reliability requirements necessary for deployment in security-critical software development environments.

While our experiments treated RAG as a standalone enhancement, future work could explore its integration with traditional static analysis workflows. Given the observed performance limitations, RAG-enhanced LLMs are better suited as complementary decision-support tools rather than primary detectors. In particular, the persistent false positive behavior observed in our study is consistent with prior findings on static analysis tools, which have been shown to repeatedly flag the same vulnerability even after a correct fix has been applied [41]. From a usability perspective, RAG-enhanced LLMs may therefore be most valuable in providing contextual explanations or supporting the triage and prioritization of findings produced by existing non-dynamic analysis instruments, rather than replacing them.

We further observe that differences across LLMs reflect model-level effects, while factors such as retrieval sparsity and dataset composition arise from the evaluation data, and Java-specific coding idioms and fix patterns influence language-level behavior.

Our study should be understood as an empirical exploration of LLM behavior under RAG-style augmentation, rather than as a practical solution for real-world deployment.

REFERENCES

[1] A. Aggarwal and P. Jalote, "Integrating static and dynamic analysis for detecting vulnerabilities," in *Proc. 30th Annu. Int. Comput. Softw. Appl. Conf. (COMPSAC)*, Sep. 2006, pp. 343–350.

[2] V. Akuthota, R. Kasula, S. T. Sumona, M. Mitul, M. T. Reza, and M. D. Rahman, "Vulnerability detection and monitoring using llm, "Vulnerability detection and monitoring using LLM," in *Proc. IEEE 9th Int. Women Eng. (WIE) Conf. Elect. Comput. Eng. (WIECON-ECE)*, Nov. 2023, pp. 309–314.

[3] R. A. Majizi, H. Shaker, B. Kumar, and Z. T. Sharef, "Vulnerability detection: Dynamic analysis of web applications and assessment of penetration testing tools," in *AI and IoT: Driving Business Success and Sustainability in the Digital Age*, vol. 2. Switzerland: Springer, 2025, pp. 801–811. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-88874-8_71

[4] A. Arusoaie, S. Ciobâca, V. Craciun, D. Gavrilut, and D. Lucanu, "A comparison of open-source static analysis tools for vulnerability detection in C/C++ code," in *Proc. 19th Int. Symp. Symbolic Numeric Algorithms Scientific Comput. (SYNASC)*, Sep. 2017, pp. 161–168.

[5] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software," in *Proc. 17th Int. Conf. Predictive Models Data Analytics Softw. Eng.*, Aug. 2021, pp. 30–39.

[6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, and A. Askell, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[7] Q.-C. Bui, R. Scandariato, and N. E. D. Ferreyra, "Vul4J: A dataset of reproducible Java vulnerabilities geared towards the study of program repair techniques," in *Proc. IEEE/ACM 19th Int. Conf. Mining Softw. Repositories (MSR)*, May 2022, pp. 464–468.

[8] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022.

[9] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2023, pp. 654–668.

[10] E. Collini, F. Indra Kurniadi, P. Nesi, and G. Pantaleo, "Context-aware retrieval augmented generation using similarity validation to handle context inconsistencies in large language models," *IEEE Access*, vol. 13, pp. 170065–170080, 2025.

[11] *Cybernative/code_vulnerability_security_dpo–Datasets At Hugging Face*, CyberNative AI LLC, Sacramento, CA, USA, 2024, Accessed: Oct. 12, 2025.

[12] X. Du, G. Zheng, K. Wang, Y. Zou, Y. Wang, W. Deng, J. Feng, M. Liu, B. Chen, X. Peng, T. Ma, and Y. Lou, "Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG," 2024, *arXiv:2406.11147*.

[13] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.

[14] Z. Gao, H. Wang, Y. Zhou, W. Zhu, and C. Zhang, "How far have we gone in vulnerability detection using large language models," 2023, *arXiv:2311.12420*.

[15] K. Goseva-Popstojanova and A. Perhinschi, "On the capability of static code analysis to detect security vulnerabilities," *Inf. Softw. Technol.*, vol. 68, pp. 18–33, Dec. 2015.

[16] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang, "Deepseek-coder: When the large language model meets programming—The rise of code intelligence," 2024, *arXiv:2401.14196*.

[17] Y. Guo, C. Patsakis, Q. Hu, Q. Tang, and F. Casino, "Outside the comfort zone: Analysing LLM capabilities in software vulnerability detection," in *Proc. Comput. Secur. - ESORICS : 29th Eur. Symp. Res. Comput. Secur.*, Bydgoszcz, Poland. Berlin, Germany: Springer, Sep. 2024, pp. 271–289.

[18] R. Heumüller, T. Langer, and F. Ortmeier, "Empirical analysis of openai embeddings for semantic code review comment similarity," in *Proc. Euromicro Conf. Softw. Eng. Adv. Appl.* Springer, Sep. 2025, pp. 37–45.

[19] Y. Jiao, J. Han, and C. Huang, "DeepVulHunter: Enhancing the code vulnerability detection capability of LLMs through multi-round analysis," *J. Intell. Inf. Syst.*, vol. 63, no. 6, pp. 2237–2264, Dec. 2025.

[20] S. Kaniewski, F. Schmidt, M. Enzweiler, M. Menth, and T. Heer, "A systematic literature review on detecting software vulnerabilities with large language models," 2025, *arXiv:2507.22659*.

[21] I. Kazemian, P. Ramanan, and M. Yildirim, "Text embedding models can be great data engineers," 2025, *arXiv:2505.14802*.

[22] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," in *Proc. IEEE Conf. Softw. Testing, Verification Validation (ICST)*, Mar. 2025, pp. 103–114.

[23] L. Kumar, V. Singh, S. Patel, and P. Mishra, "Empowering sw security: Codebert and machine learning approaches to vulnerability detection," in *Proc. 21st Int. Conf. Natural Lang. Process. (ICON)*, 2024, pp. 399–407.

[24] T. H. M. Le, M. A. Babar, and T. H. Thai, "Software vulnerability prediction in low-resource languages: An empirical study of CodeBERT and ChatGPT," in *Proc. 28th Int. Conf. Eval. Assessment Softw. Eng.*, Jun. 2024, pp. 679–685.

[25] H. Li, H. Yu, Y. Zhai, and Z. Qian, "Enhancing static analysis for practical bug detection: An LLM-integrated approach," in *Proc. ACM Program. Lang.*, Apr. 2024, vol. 8, no. OOPSLA1, pp. 474–499.

[26] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A deep learning-based system for vulnerability detection," 2018, *arXiv:1801.01681*.

[27] S. Lipp, S. Banescu, and A. Pretschner, "An empirical study on the effectiveness of static c code analyzers for vulnerability detection," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Test. Anal.* New York, NY, USA: Association for Computing Machinery, Jul. 2022, pp. 544–555.

[28] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.

[29] I. Medeiros, N. Neves, and M. Correia, "Detecting and removing web application vulnerabilities with static analysis and data mining," *IEEE Trans. Rel.*, vol. 65, no. 1, pp. 54–69, Mar. 2016.

[30] R. Mim, A. Satter, T. Ahammed, and K. Sakib, "Automated software vulnerability detection using CodeBERT and convolutional neural network," in *Proc. 19th Int. Conf. Eval. Novel Approaches to Softw. Eng.*, 2024, pp. 156–167.

[31] C. Ni, L. Shen, X. Yang, Y. Zhu, and S. Wang, "MegaVul: A C/C++ vulnerability dataset with comprehensive code representations," in *Proc. 21st Int. Conf. Mining Softw. Repositories*, Apr. 2024, pp. 738–742.

[32] M. Paterson and V. Dančík, "Longest common subsequences," in *Mathematical Foundations of Computer Science*, I. Prívara, B. Rovan, and P. Ruzička, Eds., Berlin, Germany: Springer, 1994, pp. 127–142.

[33] I. Radeva, I. Popchev, and M. Dimitrova, "Similarity thresholds in retrieval-augmented generation," in *Proc. IEEE 12th Int. Conf. Intell. Syst. (IS)*, Aug. 2024, pp. 1–7.

[34] S. Rawat, D. Ceara, L. Mounier, and M.-L. Potet, "Combining static and dynamic analysis for vulnerability detection," 2013, *arXiv:1305.3883*.

[35] N. Risse and M. Böhme, "Uncovering the limits of machine learning for automatic vulnerability detection," in *Proc. 33rd USENIX Secur. Symp. (USENIX Secur. 24)*, 2023, pp. 4247–4264.

[36] R. Safdar, D. Mateen, S. Taha Ali, and W. Hussain, "Real-VulLLM: An LLM based assessment framework in the wild," 2025, *arXiv:2510.04056*.

[37] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.

[38] A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, P. Zadorozhny, A. Cheshkov, R. Mussabayev, A. Toleu, G. Tolegen, and A. Krassovitskiy, "Finetuning large language models for vulnerability detection," *IEEE Access*, vol. 13, pp. 38889–38900, 2025.

[39] K. Tsipenyuk, B. Chess, and G. McGraw, "Seven pernicious kingdoms: A taxonomy of software security errors," *IEEE Secur. Privacy Mag.*, vol. 3, no. 6, pp. 81–84, Nov. 2005.

[40] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 862–880.

[41] T. Viszkok and P. Hegedűs, "Unified SAST benchmark: Compare AI-driven and traditional static analyzers," *SoftwareX*, vol. 31, Sep. 2025, Art. no. 102336.

[42] W. Wang, T. N. Nguyen, S. Wang, Y. Li, J. Zhang, and A. Yadavally, "DeepVD: Toward class-separation features for neural network vulnerability detection," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, May 2023, pp. 2249–2261.

[43] Y. Wang, H. Le, A. Deepak Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, "CodeT5+: Open code large language models for code understanding and generation," 2023, *arXiv:2305.07922*.

[44] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder–decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.

[45] C. Wen, Y. Cai, B. Zhang, J. Su, Z. Xu, D. Liu, S. Qin, Z. Ming, and T. Cong, "Automatically inspecting thousands of static bug warnings with large language model: How far are we?" *ACM Trans. Knowl. Discovery from Data*, vol. 18, no. 7, pp. 1–34, Jun. 2024.

[46] Y. Xia, H. Shao, and X. Deng, "VulCoBERT: A CodeBERT-based system for source code vulnerability detection," in *Proc. Int. Conf. Generative Artif. Intell. Inf. Secur.*, May 2024, pp. 249–252.

[47] Y. Xu, M. Zhang, X. Wang, J. Chen, R. Liang, Y. Zhen, and C. Zhen, "A review of code vulnerability detection techniques based on static analysis," in *Proc. Int. Conf. Comput. & Experim. Eng. Sci.* Switzerland: Springer, May 2023, pp. 251–272.

[48] A. Z. H. Yang, C. Le Goues, R. Martins, and V. Hellendoorn, "Large language models for test-free fault localization," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Feb. 2024, pp. 1–12.

[49] M.-J. Yoon, S.-M. Yoo, J.-H. Park, and K.-W. Park, "Implementation of multi-level RAG model for enhanced synergistic vulnerability analysis," in *Proc. 1st Int. Conf. Consum. Technol. (ICCT-Pacific)*, Mar. 2025, pp. 1–4.

[50] O. Zaazaa and H. El Bakkali, "Dynamic vulnerability detection approaches and tools: State of the art," in *Proc. 4th Int. Conf. Intell. Comput. Data Sci. (ICDS)*, Oct. 2020, pp. 1–6.

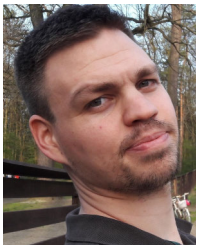
[51] K. Zhao, S. Duan, G. Qiu, J. Zhai, M. Li, and L. Liu, "Python source code vulnerability detection based on CodeBERT language model," in *Proc. 7th Int. Conf. Algorithms, Comput. Artif. Intell. (ACAI)*, Dec. 2024, pp. 1–6.
[52] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Trans. Softw. Eng. Methodology*, vol. 34, no. 5, pp. 1–31, Jun. 2025.
[53] X. Zhou, T. Zhang, and D. Lo, "Large language model for vulnerability detection: Emerging results and future directions," in *Proc. ACM/IEEE 44th Int. Conf. Softw. Eng., New Ideas Emerging Results*, May 2024, pp. 47–51.
[54] Y. Zhou, S. Liu, J. K. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 10197–10207.

GÁBOR ANTAL received the Ph.D. degree in computer science from the University of Szeged, in 2022.

He is currently an Assistant Professor with the Department of Software Engineering, University of Szeged. Besides teaching and research activities, he is a project manager and a lead developer on various research and development projects. His areas of interests include static analysis of dynamic languages, deep learning applications, vulnerability and bug detection, and large language models. Numerous publications can be associated with him, with more than 300 citations. He served as a PC Member for many conferences (e.g., MSR, PROMISE, and SCAM).

LÓRÁNT ÉRTEKES is currently pursuing the bachelor's degree in computer science with the University of Szeged. He is with the Department of Software Engineering, University of Szeged. His areas of interests include vulnerability detection and large language models.

NORBERT SZOLNOKI received the master's degree in software engineering, in 2024. He is currently pursuing the Ph.D. degree with the University of Szeged. His research interests include software engineering and software security, with a particular focus on source code vulnerability detection, static and dynamic program analysis, and machine learning–based analysis of software systems. Alongside his doctoral studies, he works as a senior solutions engineer, designing and implementing custom software solutions and integrations using Python, C#, .NET, and cloud-based technologies. His professional experience includes building scalable APIs, automation tools, data-processing pipelines, and developing research-oriented prototypes that bridge academic methods with real-world software systems. This combination of academic research and industry practice directly informs his research, ensuring practical applicability of his approaches to software security and program analysis.



PÉTER HEGEDŰS received the Ph.D. degree in computer science from the University of Szeged, in 2015. He is currently an Assistant Professor at the Software Engineering Department, University of Szeged, and a Researcher at FrontEndART Ltd. Besides teaching and research involvement, he also takes part in various software development projects as a project manager and a lead developer. His research interests include software maintainability models, deep learning applications, source code analysis, vulnerability detection, and prediction. His publication record consists of over 80 papers that appeared in top conferences and high-impact journals. He received various awards and scholarships during his career, such as the prestigious Bolyai János Research Scholarship. He was a PC Member of ICSE, ICSME, SANER, CSMR, MSR, QUATIC, ESEM, and SCAM conferences.